Tyler Vander Mooren

ITC 413: NoSQL Databases

Professor Bryan Knueppel

March 17, 2026

Assignment 10

Part 1: Expand Your Music Library

Screenshot 1. New songs added to the music library

```
 artist_name          | album_title                | song_title                | genre           | release_year
----------------------+----------------------------+---------------------------+-----------------+--------------
       Kendrick Lamar |                      DAMN. |                   HUMBLE. |         hip-hop |         2017
       Kendrick Lamar | Mr. Morale & the Big Steppers |                      N95 |         hip-hop |         2022
           The Weeknd |                After Hours |            Blinding Lights |       electronic |         2020
           Pink Floyd |       Dark Side of the Moon |                     Money | progressive rock |         1973
           Pink Floyd |       Dark Side of the Moon |                      Time | progressive rock |         1973
          Johnny Cash |           At Folsom Prison |         Folsom Prison Blues |         country |         1968
       Chris Stapleton |                 Traveller |          Tennessee Whiskey |         country |         2015
           The Beatles |                Abbey Road |          Here Comes the Sun |            rock |         1969
           The Beatles |                   Let It Be |                   Let It Be |            rock |         1970
       Michael Jackson |                  Thriller |                Billie Jean |             pop |         1982
          Miles Davis |                Kind of Blue |              Blue in Green |            jazz |         1959
          Miles Davis |                Kind of Blue |                   So What |            jazz |         1959
  Ludwig van Beethoven |             Symphony No. 5 | Symphony No. 5 in C Minor |       classical |         1960
                N.W.A |      Straight Outta Compton |     Straight Outta Compton |         hip-hop |         1988
               Eminem |      The Marshall Mathers LP |        The Real Slim Shady |         hip-hop |         2000
                Queen |           Bohemian Rhapsody |          Bohemian Rhapsody |            rock |         1975
            Daft Punk |                  Discovery |              One More Time |       electronic |         2000
          Taylor Swift |                       1989 |                Blank Space |             pop |         2014
               Nirvana |                  Nevermind |    Smells Like Teen Spirit |           grunge |         1991

(19 rows)
token@cqlsh:music_library>
token@cqlsh:music_library> SELECT * FROM songs_by_genre WHERE genre = 'hip-hop';

 genre   | release_year | artist_name    | song_title            | album_title                  | duration_seconds
---------+--------------+----------------+-----------------------+------------------------------+------------------
 hip-hop |         2022 | Kendrick Lamar |                   N95 | Mr. Morale & the Big Steppers |              195
 hip-hop |         2000 |         Eminem |     The Real Slim Shady |       The Marshall Mathers LP |              284
 hip-hop |         1988 |          N.W.A | Straight Outta Compton |        Straight Outta Compton |              258

(3 rows)
token@cqlsh:music_library> SELECT * FROM songs_by_genre WHERE genre = 'classical';

 genre     | release_year | artist_name          | song_title                | album_title    | duration_seconds
-----------+--------------+----------------------+---------------------------+----------------+------------------
 classical |         1960 | Ludwig van Beethoven | Symphony No. 5 in C Minor | Symphony No. 5 |              425
```

The first screenshot shows the expanded song collection after the new records were inserted.

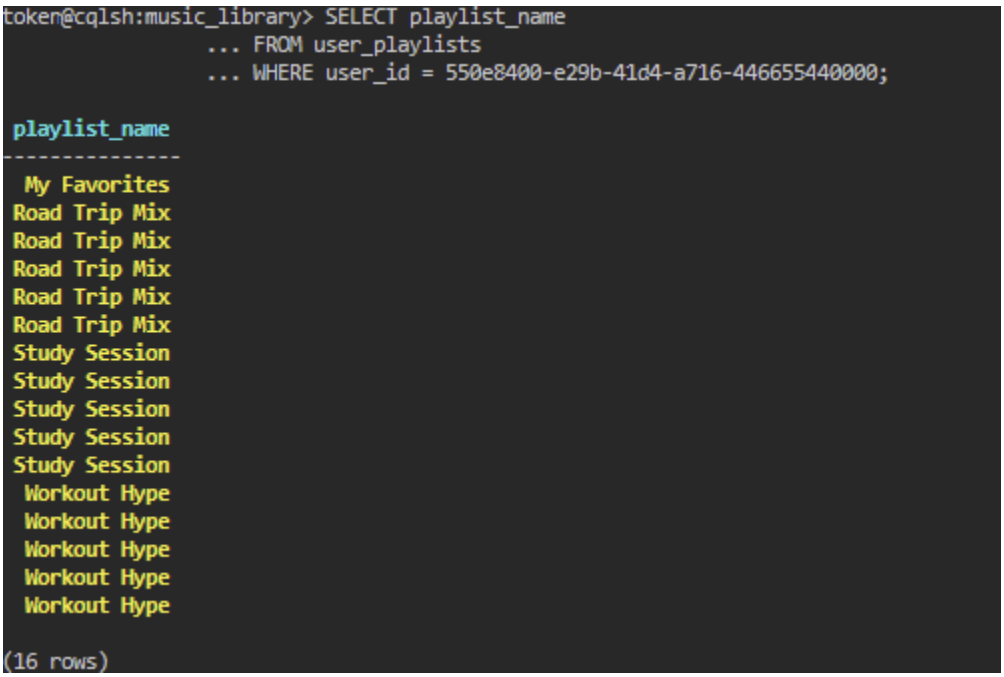
Screenshot 2. Songs from different genres and songs from different decades

This screenshot shows songs retrieved by genre, such as hip-hop, classical, electronic and country; this demonstrates that the songs_by_genre table supports direct genre-based lookups efficiently. Also, in the second part of the screenshot, it shows songs from different decades which confirms that the data set now spans from the 1960s to the 2020s. This supports broader filtering and reporting across time periods.

Part 2: Build a Playlist Management System

I used the user_playlists table to create three playlists for the user which are Road Trip Mix, Study Session and Workout Hype. Each playlist contains five songs. Because Cassandra stores rows by partition and clustering keys, playlist order was managed through the song_position column.

Screenshot 3. Three user playlists



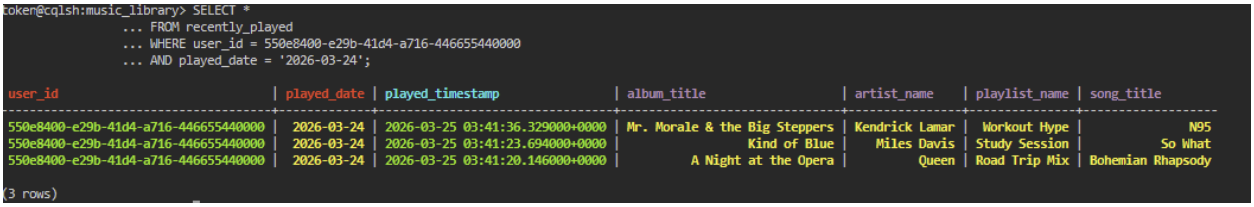
This screenshot shows the three playlists with multiple rows being returned, this is because of how Cassandra stores the data per song entry and doesn't support DISTINCT on clustering columns.

Screenshot 4. Reordered playlist

This screenshot shows the reordered version of Road Trip Mix after swapping the first two songs. Since song_position is part of the primary key, reordering required deleting and

reinserting rows with updated positions. I also created a recently_played table to track the songs the user listened to most recently. This table was modeled using user_id and played_date as the partition key and played_timestamp as the clustering column so results would be returned in reverse chronological order.

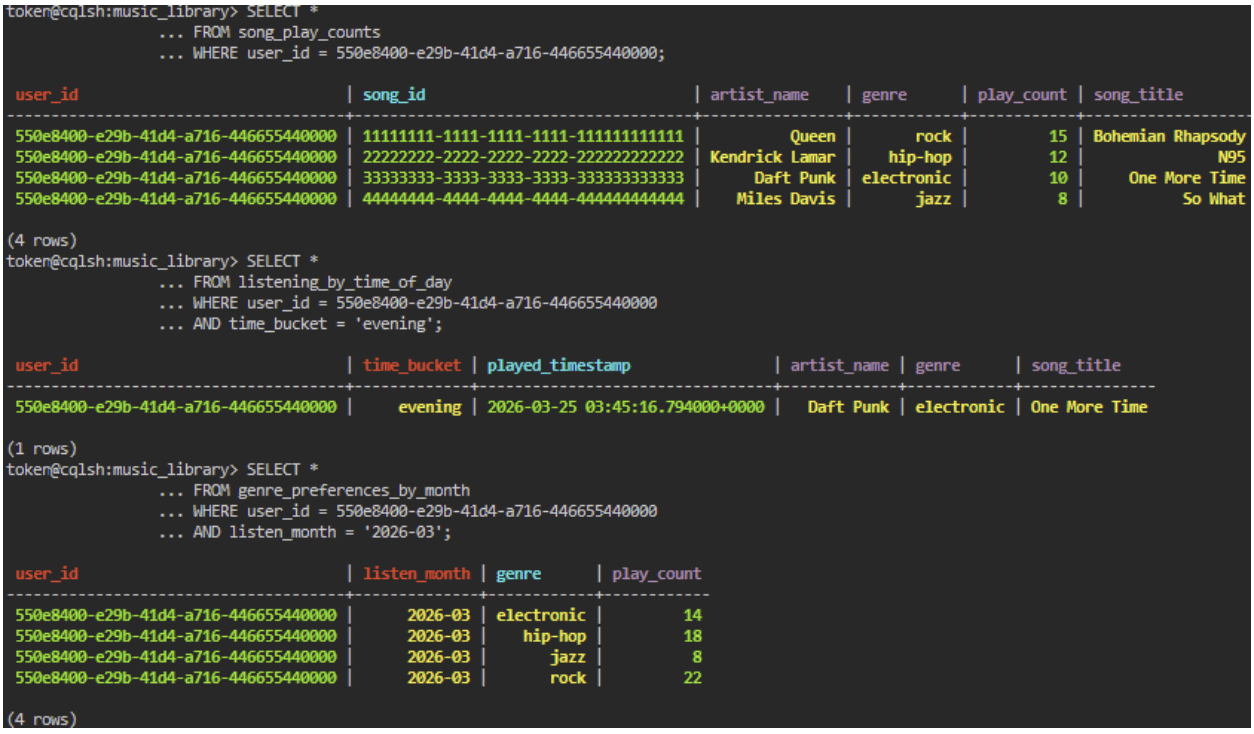
Screenshot 5. Recently played table



```
token@cqlsh:music_library> SELECT *
               ... FROM recently_played
               ... WHERE user_id = 550e8400-e29b-41d4-a716-446655440000
               ... AND played_date = '2026-03-24';

 user_id                              | played_date | played_timestamp                | album_title                | artist_name    | playlist_name | song_title
--------------------------------------+-------------+---------------------------------+----------------------------+----------------+---------------+-----------------
 550e8400-e29b-41d4-a716-446655440000 |  2026-03-24 | 2026-03-25 03:41:36.329000+0000 | Mr. Morale & the Big Steppers | Kendrick Lamar |  Workout Hype |             N95
 550e8400-e29b-41d4-a716-446655440000 |  2026-03-24 | 2026-03-25 03:41:23.694000+0000 |            Kind of Blue |    Miles Davis | Study Session |          So What
 550e8400-e29b-41d4-a716-446655440000 |  2026-03-24 | 2026-03-25 03:41:20.146000+0000 |         A Night at the Opera |         Queen | Road Trip Mix | Bohemian Rhapsody

(3 rows)
```

Part 3: Music Analytics and Reporting

Screenshot 6. Listening statistics tables and results



```
token@cqlsh:music_library> SELECT *
               ... FROM song_play_counts
               ... WHERE user_id = 550e8400-e29b-41d4-a716-446655440000;

 user_id                              | song_id                              | artist_name    | genre      | play_count | song_title
--------------------------------------+--------------------------------------+----------------+------------+------------+-----------------
 550e8400-e29b-41d4-a716-446655440000 | 11111111-1111-1111-1111-111111111111 |         Queen |       rock |         15 | Bohemian Rhapsody
 550e8400-e29b-41d4-a716-446655440000 | 22222222-2222-2222-2222-222222222222 | Kendrick Lamar |    hip-hop |         12 |             N95
 550e8400-e29b-41d4-a716-446655440000 | 33333333-3333-3333-3333-333333333333 |      Daft Punk | electronic |         10 |     One More Time
 550e8400-e29b-41d4-a716-446655440000 | 44444444-4444-4444-4444-444444444444 |    Miles Davis |       jazz |          8 |          So What

(4 rows)
token@cqlsh:music_library> SELECT *
               ... FROM listening_by_time_of_day
               ... WHERE user_id = 550e8400-e29b-41d4-a716-446655440000
               ... AND time_bucket = 'evening';

 user_id                              | time_bucket | played_timestamp                | artist_name | genre      | song_title
--------------------------------------+-------------+---------------------------------+-------------+------------+-------------
 550e8400-e29b-41d4-a716-446655440000 |     evening | 2026-03-25 03:45:16.794000+0000 |   Daft Punk | electronic | One More Time

(1 rows)
token@cqlsh:music_library> SELECT *
               ... FROM genre_preferences_by_month
               ... WHERE user_id = 550e8400-e29b-41d4-a716-446655440000
               ... AND listen_month = '2026-03';

 user_id                              | listen_month | genre      | play_count
--------------------------------------+--------------+------------+------------
 550e8400-e29b-41d4-a716-446655440000 |      2026-03 | electronic |         14
 550e8400-e29b-41d4-a716-446655440000 |      2026-03 |    hip-hop |         18
 550e8400-e29b-41d4-a716-446655440000 |      2026-03 |       jazz |          8
 550e8400-e29b-41d4-a716-446655440000 |      2026-03 |       rock |         22

(4 rows)
```

Instead of trying to calculate everything from one base table, each table was designed around a specific reporting query. The song_play_counts table tracks the most played songs for a user. In my sample data, Bohemian Rhapsody had the highest play count, followed by N95, One

More Time, and So What. This provides a direct answer to the “most played songs” requirement. The listening_by_time_of_day table groups listening behavior into buckets such as morning, afternoon, evening, and night. The genre_preferences_by_month table tracks how often a user plays genres over time. In my sample data for March 2026, rock had the highest play count, followed by hip-hop, electronic and jazz.

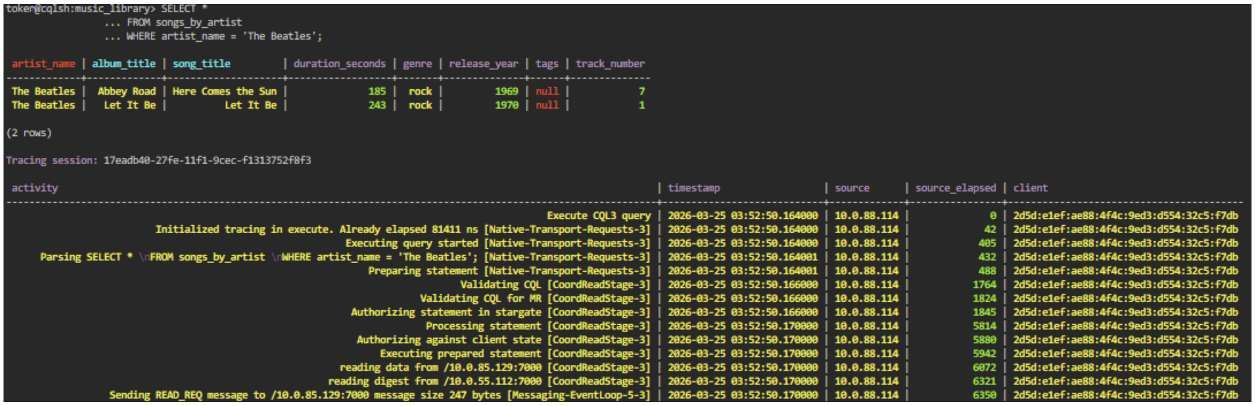
Part 4: Music Recommendation Engine Data Model

Screenshot 7. Song ratings and similar artists tables

I designed two tables to support a basic recommendation system with user_song_ratings and similar_artists. The first stores song ratings by user and genre, while the second stores artist-to-artist similarity relationships. The user_song_ratings table supports recommendations based on the user’s own tastes. For example, highly rated songs in rock, hip-hop, electronic and country can be used to recommend more songs from those genres. The similar_artists table supports artist-based recommendations by linking artists such as Queen to The Beatles and Pink Floyd or Kendrick Lamar to Eminem.

Part 5: Performance Testing and Optimization

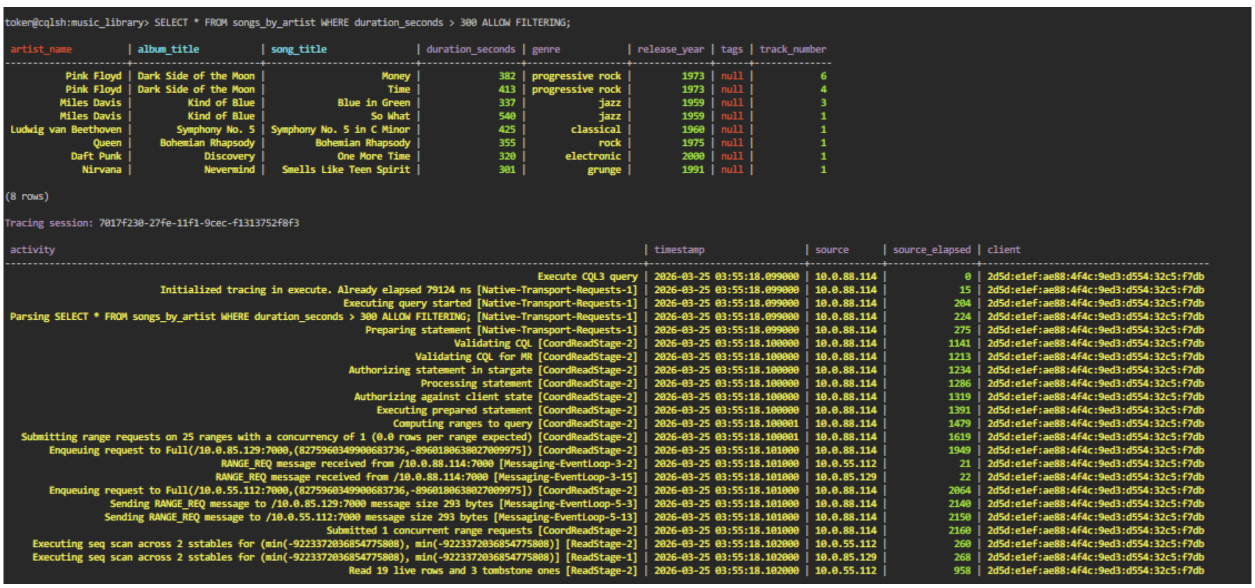
Screenshot 8. Efficient traced query



```
token@cqlsh:music_library> SELECT *
              ... FROM songs_by_artist
              ... WHERE artist_name = 'The Beatles';

 artist_name | album_title | song_title          | duration_seconds | genre | release_year | tags | track_number
-------------+-------------+---------------------+------------------+-------+--------------+------+-------------
 The Beatles |  Abbey Road | Here Comes the Sun |              185 |  rock |         1969 | null |            7
 The Beatles |  Let It Be  |          Let It Be |              243 |  rock |         1970 | null |            1

(2 rows)

Tracing session: 17eadb40-27fe-11f1-9cec-f1313752f8f3

 activity                                                                                              | timestamp                  | source      | source_elapsed | client
-------------------------------------------------------------------------------------------------------+----------------------------+-------------+----------------+-----------------------------------------
                                                                                    Execute CQL3 query | 2026-03-25 03:52:50.164000 | 10.0.88.114 |              0 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                         Initialized tracing in execute. Already elapsed 81411 ns [Native-Transport-Requests-3] | 2026-03-25 03:52:50.164000 | 10.0.88.114 |             42 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                          Executing query started [Native-Transport-Requests-3] | 2026-03-25 03:52:50.164000 | 10.0.88.114 |            405 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
   Parsing SELECT * \nFROM songs_by_artist \nWHERE artist_name = 'The Beatles'; [Native-Transport-Requests-3] | 2026-03-25 03:52:50.164001 | 10.0.88.114 |            432 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                              Preparing statement [Native-Transport-Requests-3] | 2026-03-25 03:52:50.164001 | 10.0.88.114 |            488 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                                         Validating CQL [CoordReadStage-3] | 2026-03-25 03:52:50.166000 | 10.0.88.114 |           1764 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                                  Validating CQL for MR [CoordReadStage-3] | 2026-03-25 03:52:50.166000 | 10.0.88.114 |           1824 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                      Authorizing statement in stargate [CoordReadStage-3] | 2026-03-25 03:52:50.166000 | 10.0.88.114 |           1845 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                                   Processing statement [CoordReadStage-3] | 2026-03-25 03:52:50.170000 | 10.0.88.114 |           5814 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                      Authorizing against client state [CoordReadStage-3] | 2026-03-25 03:52:50.170000 | 10.0.88.114 |           5880 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                          Executing prepared statement [CoordReadStage-3] | 2026-03-25 03:52:50.170000 | 10.0.88.114 |           5942 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                   reading data from /10.0.85.129:7000 [CoordReadStage-3] | 2026-03-25 03:52:50.170000 | 10.0.88.114 |           6072 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                 reading digest from /10.0.55.112:7000 [CoordReadStage-3] | 2026-03-25 03:52:50.170000 | 10.0.88.114 |           6321 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
          Sending READ_REQ message to /10.0.85.129:7000 message size 247 bytes [Messaging-EventLoop-5-3] | 2026-03-25 03:52:50.170000 | 10.0.88.114 |           6350 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
```

The efficient query was: SELECT * FROM songs_by_artist WHERE artist_name = 'The Beatles'; this query used the partition key directly, so Cassandra was able to retrieve the rows with relatively little work.

Screenshot 9. Inefficient traced queries



```
token@cqlsh:music_library> SELECT * FROM songs_by_artist WHERE duration_seconds > 300 ALLOW FILTERING;

 artist_name          | album_title          | song_title                | duration_seconds | genre             | release_year | tags | track_number
----------------------+----------------------+---------------------------+------------------+-------------------+--------------+------+-------------
           Pink Floyd | Dark Side of the Moon |                     Money |              382 | progressive rock |         1973 | null |            6
           Pink Floyd | Dark Side of the Moon |                      Time |              413 | progressive rock |         1973 | null |            4
          Miles Davis |          Kind of Blue |             Blue in Green |              337 |              jazz |         1959 | null |            3
          Miles Davis |          Kind of Blue |                   So What |              540 |              jazz |         1959 | null |            1
 Ludwig van Beethoven |        Symphony No. 5 | Symphony No. 5 in C Minor |              425 |         classical |         1960 | null |            1
                Queen |     Bohemian Rhapsody |         Bohemian Rhapsody |              355 |              rock |         1975 | null |            1
            Daft Punk |             Discovery |             One More Time |              320 |        electronic |         2000 | null |            1
              Nirvana |             Nevermind |  Smells Like Teen Spirit |              301 |            grunge |         1991 | null |            1

(8 rows)

Tracing session: 7017f230-27fe-11f1-9cec-f1313752f8f3

 activity                                                                                              | timestamp                  | source      | source_elapsed | client
-------------------------------------------------------------------------------------------------------+----------------------------+-------------+----------------+-----------------------------------------
                                                                                    Execute CQL3 query | 2026-03-25 03:55:18.099000 | 10.0.88.114 |              0 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                         Initialized tracing in execute. Already elapsed 79124 ns [Native-Transport-Requests-1] | 2026-03-25 03:55:18.099000 | 10.0.88.114 |             15 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                          Executing query started [Native-Transport-Requests-1] | 2026-03-25 03:55:18.099000 | 10.0.88.114 |            204 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Parsing SELECT * FROM songs_by_artist WHERE duration_seconds > 300 ALLOW FILTERING; [Native-Transport-Requests-1] | 2026-03-25 03:55:18.099000 | 10.0.88.114 |            224 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                              Preparing statement [Native-Transport-Requests-1] | 2026-03-25 03:55:18.099000 | 10.0.88.114 |            275 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                                         Validating CQL [CoordReadStage-2] | 2026-03-25 03:55:18.100000 | 10.0.88.114 |           1141 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                                  Validating CQL for MR [CoordReadStage-2] | 2026-03-25 03:55:18.100000 | 10.0.88.114 |           1213 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                      Authorizing statement in stargate [CoordReadStage-2] | 2026-03-25 03:55:18.100000 | 10.0.88.114 |           1234 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                                   Processing statement [CoordReadStage-2] | 2026-03-25 03:55:18.100000 | 10.0.88.114 |           1286 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                      Authorizing against client state [CoordReadStage-2] | 2026-03-25 03:55:18.100000 | 10.0.88.114 |           1319 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                          Executing prepared statement [CoordReadStage-2] | 2026-03-25 03:55:18.100000 | 10.0.88.114 |           1391 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                             Computing ranges to query [CoordReadStage-2] | 2026-03-25 03:55:18.100001 | 10.0.88.114 |           1479 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
  Submitting range requests on 25 ranges with a concurrency of 1 (0.0 rows per range expected) [CoordReadStage-2] | 2026-03-25 03:55:18.100001 | 10.0.88.114 |           1619 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
     Enqueuing request to Full(/10.0.85.129:7000,(8275960349900683736,-8960180638027009975]) [CoordReadStage-2] | 2026-03-25 03:55:18.101000 | 10.0.88.114 |           1949 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                      RANGE_REQ message received from /10.0.88.114:7000 [Messaging-EventLoop-3-2] | 2026-03-25 03:55:18.101000 | 10.0.55.112 |             21 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                     RANGE_REQ message received from /10.0.88.114:7000 [Messaging-EventLoop-3-15] | 2026-03-25 03:55:18.101000 | 10.0.85.129 |             22 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
     Enqueuing request to Full(/10.0.55.112:7000,(8275960349900683736,-8960180638027009975]) [CoordReadStage-2] | 2026-03-25 03:55:18.101000 | 10.0.88.114 |           2064 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
           Sending RANGE_REQ message to /10.0.85.129:7000 message size 293 bytes [Messaging-EventLoop-5-3] | 2026-03-25 03:55:18.101000 | 10.0.88.114 |           2140 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
          Sending RANGE_REQ message to /10.0.55.112:7000 message size 293 bytes [Messaging-EventLoop-5-13] | 2026-03-25 03:55:18.101000 | 10.0.88.114 |           2150 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                   Submitted 1 concurrent range requests [CoordReadStage-2] | 2026-03-25 03:55:18.101000 | 10.0.88.114 |           2160 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
   Executing seq scan across 2 sstables for (min(-9223372036854775808), min(-9223372036854775808)] [ReadStage-2] | 2026-03-25 03:55:18.102000 | 10.0.55.112 |            260 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
   Executing seq scan across 2 sstables for (min(-9223372036854775808), min(-9223372036854775808)] [ReadStage-1] | 2026-03-25 03:55:18.102000 | 10.0.85.129 |            268 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
                                  Read 19 live rows and 3 tombstone ones [ReadStage-2] | 2026-03-25 03:55:18.102000 | 10.0.55.112 |            958 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
```

The inefficient query was: SELECT * FROM songs_by_artist WHERE duration_seconds > 300 ALLOW FILTERING; this query performed much worse because duration_seconds is not part of the primary key. The tracing output showed range requests, multiple node interactions and

broader scanning behavior, so the query also touched the most nodes and had the worst overall performance.

Screenshot 10. Range query trace

```
token@cqlsh:music_library> SELECT * FROM songs_by_genre WHERE genre = 'rock' AND release_year >= 1970 AND release_year <= 1980;

 genre | release_year | artist_name | song_title        | album_title       | duration_seconds
-------+--------------+-------------+-------------------+-------------------+------------------
  rock |         1975 |       Queen | Bohemian Rhapsody | Bohemian Rhapsody |              355
  rock |         1970 | The Beatles |         Let It Be |         Let It Be |              243

(2 rows)

Tracing session: 86d616f0-27fe-11f1-9cec-f1313752f8f3

 activity | timestamp | source | source_elapsed | client
----------------------------------------------------------
 Execute CQL3 query | 2026-03-25 03:55:56.256000 | 10.0.88.114 | 0 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Initialized tracing in execute. Already elapsed 66811 ns [Native-Transport-Requests-2] | 2026-03-25 03:55:56.255000 | 10.0.88.114 | 12 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Executing query started [Native-Transport-Requests-2] | 2026-03-25 03:55:56.256000 | 10.0.88.114 | 182 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Parsing SELECT * FROM songs_by_genre WHERE genre = 'rock' AND release_year >= 1970 AND release_year <= 1980; [Native-Transport-Requests-2] | 2026-03-25 03:55:56.256000 | 10.0.88.114 | 203 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Preparing statement [Native-Transport-Requests-2] | 2026-03-25 03:55:56.256000 | 10.0.88.114 | 260 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Validating CQL [CoordReadStage-2] | 2026-03-25 03:55:56.256001 | 10.0.88.114 | 994 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Validating CQL for MR [CoordReadStage-2] | 2026-03-25 03:55:56.256001 | 10.0.88.114 | 1038 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Authorizing statement in stargate [CoordReadStage-2] | 2026-03-25 03:55:56.257000 | 10.0.88.114 | 1061 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Processing statement [CoordReadStage-2] | 2026-03-25 03:55:56.260000 | 10.0.88.114 | 4238 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 READ_REQ message received from /10.0.88.114:7000 [Messaging-EventLoop-3-15] | 2026-03-25 03:55:56.260000 | 10.0.85.129 | 22 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Authorizing against client state [CoordReadStage-2] | 2026-03-25 03:55:56.260000 | 10.0.88.114 | 4294 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Executing prepared statement [CoordReadStage-2] | 2026-03-25 03:55:56.260000 | 10.0.88.114 | 4368 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 reading data from /10.0.85.129:7000 [CoordReadStage-2] | 2026-03-25 03:55:56.260000 | 10.0.88.114 | 4517 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 reading digest from /10.0.55.112:7000 [CoordReadStage-2] | 2026-03-25 03:55:56.260000 | 10.0.88.114 | 4611 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Sending READ_REQ message to /10.0.85.129:7000 message size 225 bytes [Messaging-EventLoop-5-3] | 2026-03-25 03:55:56.260000 | 10.0.88.114 | 4619 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Sending READ_REQ message to /10.0.55.112:7000 message size 226 bytes [Messaging-EventLoop-5-13] | 2026-03-25 03:55:56.260001 | 10.0.88.114 | 4672 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 READ_REQ message received from /10.0.88.114:7000 [Messaging-EventLoop-3-2] | 2026-03-25 03:55:56.261000 | 10.0.55.112 | 31 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Executing single-partition query on songs_by_genre; stage READ pending: 0, active: 1 [ReadStage-1] | 2026-03-25 03:55:56.261000 | 10.0.85.129 | 317 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Executing single-partition query on songs_by_genre; stage READ pending: 0, active: 1 [ReadStage-1] | 2026-03-25 03:55:56.261000 | 10.0.55.112 | 437 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Acquiring sstable references [ReadStage-1] | 2026-03-25 03:55:56.261000 | 10.0.85.129 | 429 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Acquiring sstable references [ReadStage-1] | 2026-03-25 03:55:56.261000 | 10.0.55.112 | 542 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Skipped 0/2 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-1] | 2026-03-25 03:55:56.261000 | 10.0.85.129 | 534 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Skipped 0/2 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-1] | 2026-03-25 03:55:56.261000 | 10.0.55.112 | 610 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 speculating read retry on Full(/10.0.77.25:7000,(-8960180638027009975,-7849097299957460796]) [CoordReadStage-2] | 2026-03-25 03:55:56.344000 | 10.0.88.114 | 88629 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Sending READ_REQ message to /10.0.77.25:7000 message size 225 bytes [Messaging-EventLoop-5-8] | 2026-03-25 03:55:56.344000 | 10.0.88.114 | 88722 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 READ_REQ message received from /10.0.88.114:7000 [Messaging-EventLoop-3-5] | 2026-03-25 03:55:56.345000 | 10.0.77.25 | 20 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
 Executing single-partition query on songs_by_genre; stage READ pending: 0, active: 1 [ReadStage-1] | 2026-03-25 03:55:56.345000 | 10.0.77.25 | 427 | 2d5d:e1ef:ae88:4f4c:9ed3:d554:32c5:f7db
```

The range query on songs_by_genre performed better than the ALLOW FILTERING query because it used the partition key (genre) and then filtered within the clustering range on release_year. Even so, it still involved more work than a direct partition-key lookup.

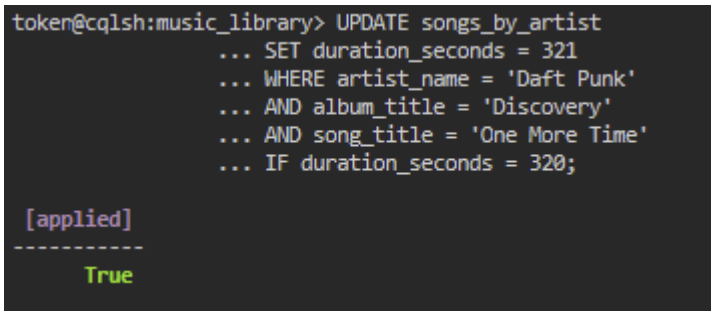
Part 6: Data Consistency and Conflict Resolution

Screenshot 11. Concurrent update and conditional insert results

```
token@cqlsh:music_library> INSERT INTO user_playlists (
               ...     user_id, playlist_name, song_position, song_id,
               ...     artist_name, song_title, album_title, added_timestamp
               ... )
               ... VALUES (
               ...     123e4567-e89b-12d3-a456-426614174000,
               ...     'Road Trip Mix',
               ...     5,
               ...     uuid(),
               ...     'Queen',
               ...     'Bohemian Rhapsody',
               ...     'A Night at the Opera',
               ...     toTimestamp(now())
               ... )
               ... IF NOT EXISTS;

 [applied]
-----------
      True

token@cqlsh:music_library> INSERT INTO user_playlists (
               ...     user_id, playlist_name, song_position, song_id,
               ...     artist_name, song_title, album_title, added_timestamp
               ... )
               ... VALUES (
               ...     123e4567-e89b-12d3-a456-426614174000,
               ...     'Road Trip Mix',
               ...     5,
               ...     uuid(),
               ...     'Nirvana',
               ...     'Smells Like Teen Spirit',
               ...     'Nevermind',
               ...     toTimestamp(now())
               ... )
               ... IF NOT EXISTS;

 [applied] | user_id                              | playlist_name | song_position | added_timestamp                 | album_title          | artist_name | song_id                              | song_title
-----------+--------------------------------------+---------------+---------------+---------------------------------+----------------------+-------------+--------------------------------------+-------------------
     False | 123e4567-e89b-12d3-a456-426614174000 | Road Trip Mix |             5 | 2026-03-25 03:58:42.907000+0000 | A Night at the Opera |       Queen | e226b8a1-128c-4f92-b2ee-cd5992baae73 | Bohemian Rhapsody
```

The first conditional insert returned True, meaning the row was written successfully. The second insert targeted the same user_id, playlist and song position, but returned False because the slot was already taken. This shows that Cassandra can prevent conflicts when conditional logic is used.

Screenshot 12. Conditional update result



Together, these results show how lightweight transactions can be used to protect shared data from accidental overwrites. The conflict resolution strategy is to use conditional writes for contested playlist positions. If two users try to write in the same position, the first successful write keeps the slot and the second write is rejected. The application can then re-read the playlist, find the next open position and retry the insert.